叶环独立集 题解

核心观察与状态

DFS 会连续访问同一子树中的所有叶子。按题目规定的孩子编号顺序合并子树时，相邻两段叶序之间只新增一条边：前一段的末叶连接后一段的首叶。因此，除了树根是否选取，只需保留这一段首叶、末叶的状态。

设 `dp[u][s][a][b]` 为以 `u` 为根的子树中的合法方案数，其中：

- `s` 表示顶点 `u` 是否选取；
- `a,b` 表示该子树 DFS 叶序的首叶、末叶是否选取；
- 子树内的树边和叶序中相邻叶子之间的边都已检查，整段首尾的闭环边暂不检查。

三个状态均取 `0/1`，每个顶点共 8 个状态，所有计数对 10^9+7 取模。

如何计算这 8 个状态

叶子 `u` 的首叶、末叶都是自身，只令 `dp[u][0][0][0]=1`、`dp[u][1][1][1]=1`，其余状态为 0。

对于非叶子，先固定 `s`，再按孩子编号从小到大合并。处理孩子 `v` 时，先求一个二维数组 `child[c][d]`：枚举孩子根状态 `t`，排除 `s=t=1`，将其余 `dp[v][t][c][d]` 相加。这一步检查树边 `(u,v)`。

第一个孩子直接赋给累积数组 `acc`。之后每接入一个孩子，枚举已有段的边界 `a,b` 和新段的边界 `c,d`；排除 `b=c=1`，执行 `merged[a][d] += acc[a][b] * child[c][d]`。这里检查的正是连接两段的叶子边，合并后首叶状态仍为 `a`，末叶状态变为 `d`。每轮都要清空 `merged`，不能在 `acc` 上原地更新。

根计算完成后，只差整棵树首尾叶子之间的闭环边。设叶子数为 `k`：当 `k=1` 时不加边，保留所有状态；当 `k>=2` 时排除 `a=b=1`。`k=2` 时闭环边与已检查的叶子边重复，但重复限制同一对端点不会改变计数。

下面的伪代码中，每个状态变量分别遍历 `{0,1}`，可直接展开成循环；所有 `+=` 和乘法结果均取模，数组赋值表示复制整个二维数组。

```
dp 全部清零
k = 没有孩子的顶点数
for u = n, n-1, ..., 1:
    if u 没有孩子:
        dp[u][0][0][0] = dp[u][1][1][1] = 1
        continue
    for s in {0,1}:
        first = true
        for v in u 的孩子（编号从小到大）:
            child[2][2] 全部清零
            for t,c,d in {0,1}:
                if not (s == 1 and t == 1):
                    child[c][d] += dp[v][t][c][d]
            if first:
                acc = child
                first = false
                continue
            merged[2][2] 全部清零
            for a,b,c,d in {0,1}:
                if not (b == 1 and c == 1):
                    merged[a][d] += acc[a][b] * child[c][d]
            acc = merged
        dp[u][s] = acc
answer = 0
for s,a,b in {0,1}:
    if k == 1 or not (a == 1 and b == 1):
        answer += dp[1][s][a][b]
输出 answer
```

迭代顺序与复杂度

输入保证 `p[u]<u`，所以按编号从 `n` 到 `1` 处理，孩子的状态一定已算好，无需递归，也不会在百万长链上爆栈。

孩子顺序必须符合题面的 DFS 顺序，不能用叶子顶点编号代替。标准程序先读父亲数组，再按 `v=n..2` 将 `v` 头插到父亲的孩子链表中，得到编号升序的孩子表；随后沿 `head/nxt` 遍历即可。

每个顶点存 8 个 32 位计数，每条树边只参与常数次状态转移，时间、空间均为 $O(n)$。标准程序使用扁平数组，连同 `parent/head/nxt` 的主要存储约 44 MB。两个模数值相加不会超过 32 位有符号整数范围，乘法应先转成 64 位。

对应实现见 std_boundary_dp.cpp。

部分分做法

- **子任务 1，$n \le 20$：** 枚举顶点子集，逐条检查树边和叶子环边，时间 $O(2^n n)$。
- **子任务 2，叶子数 $k \le 8$：** 枚举叶子选择并检查环边，再做普通树独立集 DP：父亲不选时乘各孩子两种状态之和，父亲选中时只乘孩子不选的状态。时间 $O(2^k n)$。
- **子任务 3，星形树：** 根选中贡献 1；根不选时，$m = n - 1$ 个叶子的环独立集数为 $F_{m-1} + F_{m+1}$，其中 $F_0 = 0, F_1 = 1$。时间 $O(n)$。

正确性简证

对每棵子树归纳。叶子只有选、不选两种方案，初始化恰好覆盖。假设孩子状态正确，固定 `u` 的选择后，`child` 排除了父子同时选取的情况；按叶序依次合并时，孩子内部的边已经满足限制，两个相邻叶序段之间又恰好只有末叶到首叶这一条新增边，因此只检查 `b,c` 就足够。

每个合法方案都唯一确定各孩子的选择与边界状态，而每次合法合并也唯一得到一个更大的方案，所以转移不漏计、不重计。全部孩子合并后，状态定义成立。根处按叶子数补上唯一尚未检查的首尾约束，得到的正是整张图的独立集数。

窗口校准 题解

维护原数组的一阶差分，把校准转成周期复制。下面按正解 `std_wblt.cpp` 的三类操作讲解。

所有数值运算模 $P = 998244353$，减法规范化，乘法使用 64 位中间量。下标及长度不取模，区间均为 **1 基闭区间**。

1. 校准变成差分的周期延拓

令 $a_0 = 0$，维护 $d_i = a_i - a_{i-1}$。校准 $[l, r]$、窗口长度为 k 时，设窗口和为 W_i，则

$$W_{i+1} - W_i = a_{i+k} - a_i.$$

前 $k+1$ 项不变，所以窗口公差确定为 $c = a_{l+k} - a_l$。后续需要满足 $a_{i+k} - a_i = c$，相邻两式相减得到

$$d_{i+k+1} = d_{i+1}, \qquad l \le i \le r-k-1.$$

因此，**取差分种子 $[l+1, l+k]$，周期延长到长度 $r-l$，替换差分区间 $[l+1, r]$**。注意起点是 $l+1$，d_l 不动；替换中包含种子自身，但其值不变。

直观上，原数组也可以看成块 $S, S+c, S+2c, \ldots$，其中 $S = (a_l, \ldots, a_{l+k-1})$；转成差分后就只剩周期复制。

用支持**区间截取、替换、共享倍增、区间乘、单点加，以及前缀两个矩查询**的 WBLT 维护差分即可。下文用 `slice`、`replace`、`double`、`multiply`、`add`、`prefix` 表示这些接口，`M0(T)` 表示一段差分的总和；两个矩的定义见查询部分。共享倍增无需展开实际元素。

2. 操作一：复制、截断、修正右端

设 $L = r - l$。若 $k = L$，整个区间都属于保留的前 $k+1$ 项，直接跳过。否则依次执行：

1. 截取种子 $d_{l+1}, \ldots, d_{l+k}$。
2. 不断共享倍增，直到长度至少为 L，再截取前 L 项得到 T。不能逐份拼接种子，那会使耗时依赖 L/k。
3. 计算替换前后这段差分总和的变化 δ，再替换 $[l+1, r]$。
4. 若 $r < n$，给 d_{r+1} 加 $-\delta$，抵消对区间外原数组的影响。

```
L = r-l
若 k=L：结束
T = slice(root,l+1,l+k)
while len(T) < L:
    T = double(T)
T = slice(T,1,L)
old = prefix(r).M0 - prefix(l).M0
Δ = M0(T) - old
root = replace(root,l+1,r,T)
若 r<n：add(r+1,-Δ)
```

为什么还要第 4 步：差分段总和增加 δ，会让 a_r 及右侧所有原数组值都增加 δ。补上 $d_{r+1} \leftarrow d_{r+1} - \delta$ 后，区间内保留校准结果，a_{r+1} 起恢复旧值。$r = n$ 时无须补偿。

例如 $a = (1, 3, 8, 9, 14, 20, 30)$，校准 $(l, r, k) = (1, 6, 2)$。种子为 $(2, 5)$，倍增后截取长度 5 得 $(2, 5, 2, 5, 2)$；旧段和为 19，新段和为 16，故 $\delta = -3$，需给 d_7 加 3。结果为 $(1, 3, 8, 10, 15, 17, 30)$，前 3 项和区间外的 30 均保持不变。

3. 操作二：区间乘与两个边界补偿

将 $a_l, \ldots, a_r$ 变成 $ua_i + v$。区间内部相邻两项都做同一仿射，常数项相消，所以差分只需乘 u；左右边界另外补偿。

先从旧树读取 $A = a_{l-1}$、$B = a_r$，再做任何修改。差分前缀和就是原数组单点值，故 `A=prefix(l-1).M0`、`B=prefix(r).M0`，并规定 `prefix(0)=(0,0)`。

```
A = prefix(l-1).M0
B = prefix(r).M0
w = u-1
multiply(l,r,u)
add(l,w*A+v)
若 r<n：add(r+1,-w*B-v)
```

左边界最终为 $ua_l + v - a_{l-1}$，右边界最终为 $a_{r+1} - ua_r - v$，恰好实现区间内仿射而保持外部不变。$u = 0$、$l = r$、$l = 1$ 都按同一流程处理，不能使用乘法逆元。

4. 操作三：用两个矩求原数组前缀和

对前 t 项差分，`prefix(t)` 返回

$$M_0 = \sum_{i=1}^{t} d_i, \qquad M_1 = \sum_{i=1}^{t} (i-1)d_i.$$

于是 $a_t = M_0$，原数组前缀和为

$$F(t) = \sum_{i=1}^{t} a_i = \sum_{i=1}^{t} (t-i+1)d_i = tM_0 - M_1.$$

规定 $F(0) = 0$，查询 $[l, r]$ 输出 $F(r) - F(l-1)$。注意 M_1 使用段内 0 基下标，不能直接拿绝对下标的矩套用此式。

5. 部分分

- $n, m \le 1000$：从左到右执行 $a_j = a_{j-k} + c$，仿射和查询直接遍历，时间 $O(nm)$。
- 没有操作一：用区间和、仿射懒标记线段树，时间 $O(n + m \log n)$。
- 所有 $k = 1$：校准为 $a_i \leftarrow a_l + (i-l)(a_{l+1} - a_l)$，线段树维护等差赋值与仿射，时间同上。

6. 正确性与复杂度

前两个窗口确定公差 c，后续每项满足 $a_j = a_{j-k} + c$，按从左到右的顺序唯一确定。差分周期延拓正好满足此递推，并保留前 $k+1$ 项；右端修正使区间外原数组不变。因此操作一正确。操作二的内部差分与两条边界公式分别符合目标值，操作三由交换前缀求和顺序得到，故三类操作均正确。

共享倍增每轮只新增常数个节点，截断和其余接口各需 $O(\log n)$；及时回收不用的共享节点并复用槽位后，总时间为 $\boldsymbol{O(n + m \log n)}$**（摊还）**，空间为 $\boldsymbol{O(n)}$。一次覆盖可能集中回收线性多个节点，因此不能声称每次操作最坏 $O(\log n)$。

上下左右 题解

这是一道构造题：先确定最多能走多少次 `D`，再按 N 的类型输出一条达到它的路径。下面先讲怎么构造，最优性的证明放在最后。

为便于对应代码，本文的行列坐标统一从 0 开始，起点为 $(0,0)$，终点为 $(N-1,N-1)$。**允许斜边相交，只要求格子不被重复访问**。

1. 把目标变成“少走特殊步”

记 D 为正右步数，R 为右上、右下步数之和，L 为向左的三种步数之和，V 为竖直步数。总步数与横向位移给出

$$D+R+L+V=N^2-1,\qquad D+R-L=N-1.$$

所以

$$D=\frac{N^2+N-2-C}{2},\qquad C=V+2R.$$

因此，应尽量不用右斜步，再减少竖直步。最终有四种情况：

N 的类型	构造方式	最优 D
奇数	逐行蛇形	$(N^2-1)/2$
$N\equiv 2 \pmod 4$	两列条带蛇形	$(N^2+N-2)/2$
$N=4,8$	固定小构造	$(N^2+N-4)/2$
$N\equiv 0 \pmod 4$ 且 $N\ge 12$	五阶段构造	$(N^2+N-6)/2$

2. 构造方法

2.1 奇数：逐行蛇形

第 0 行从左到右，第 1 行从右到左，交替走到最后一行；相邻行之间向下走一步。

奇数行数保证最后停在右下角。共有 $(N+1)/2$ 行从左往右，每行贡献 $N-1$ 次 `D`，总计 $(N^2-1)/2$ 次。

2.2 模 4 余 2：两列一组

将列分为 $m=N/2$ 个条带，第 j 个条带包含第 $2j,2j+1$ 列。每到一行，都先访问左格，再访问右格；然后用左上或左下走到下一行的左格。

- 偶数编号条带从上往下，奇数编号条带从下往上。
- 一条带走完后，用一次 `D` 进入下一条带。

m 为奇数，所以最后一条带向下走，恰好在右下角结束。条带内有 mN 次 `D`，条带间还有 $m-1$ 次，总计 $(N^2+N-2)/2$ 次。$N=2$ 也直接使用这个做法。

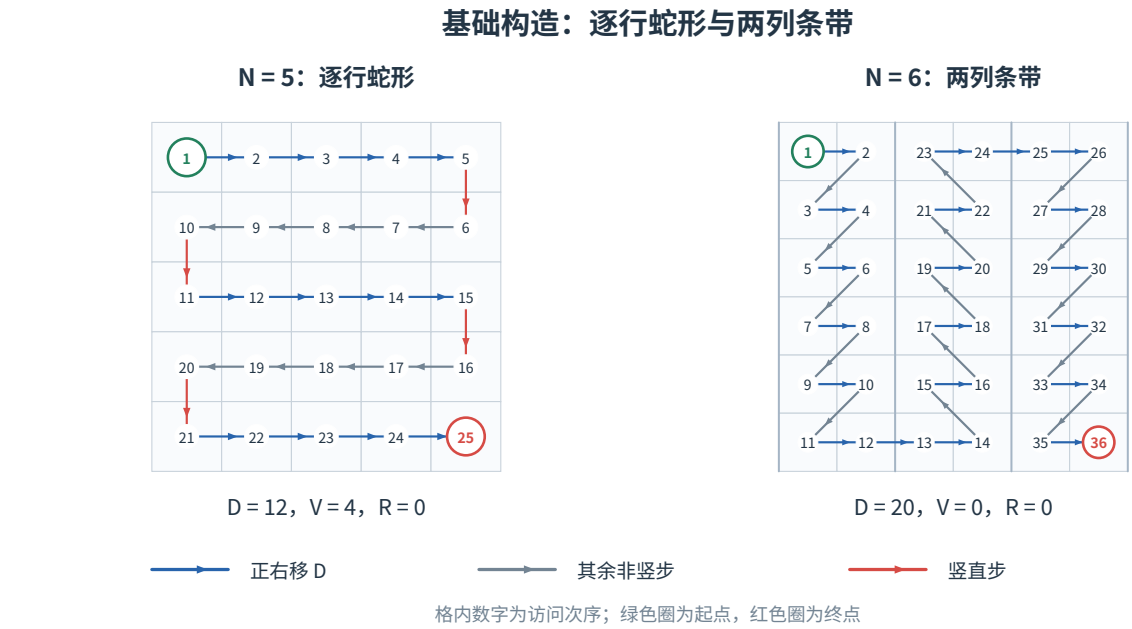


图 1　逐行蛇形与两列条带。格内数字是访问顺序，红色边是竖直步。

2.3 两个特例：$N = 4, 8$

当条带数为偶数时，上面的走法会停在右上角。$N = 4, 8$ 可以各存一条固定路径，只用两次竖直步：

```
N=4: DDDZDXAQAXDZDDD
N=8: DDDZDZDZQQDZXDZDZDZDZDDDQDQDQDDQDDQDQDQDDDZDZDZDZXZDQQZDZDZDDD
```

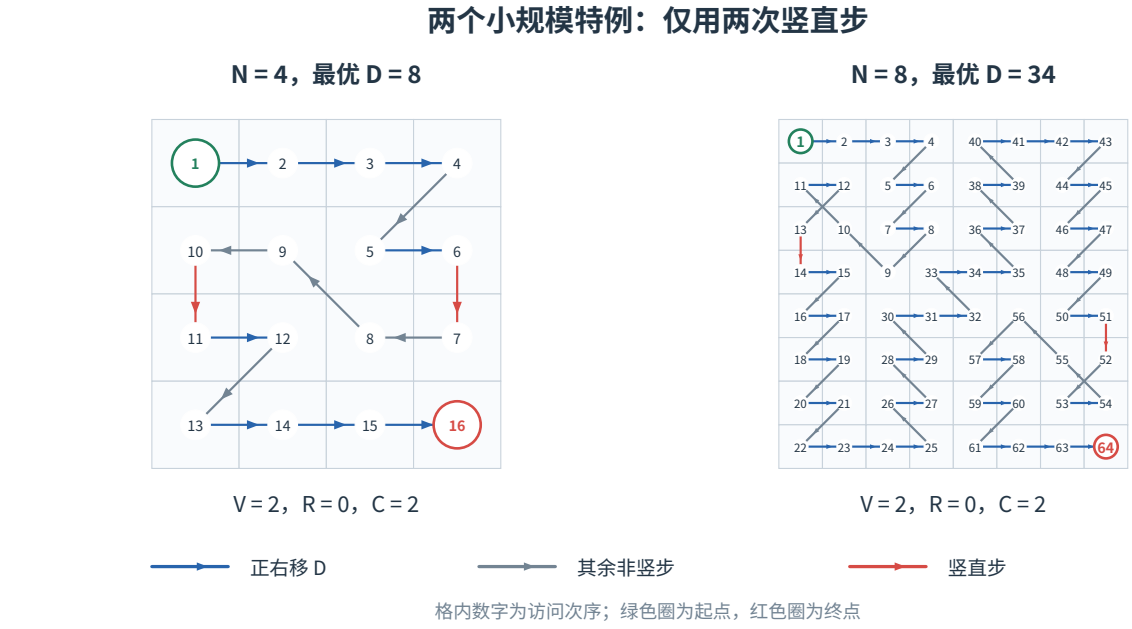


图 2　两个固定构造。按编号依次走即可，红色边给出两次竖直移动。

2.4 其余四的倍数：五个阶段、四次竖步

以下处理 $N \geq 12$、$N \equiv 0 \pmod 4$。仍然将网格分为 $m = N/2$ 个两列条带，但把路径拆成五段：

$$\text{顶部条带} \to \text{上通道} \to \text{中部条带} \to \text{下通道} \to \text{底部条带}\ .$$

三个“条带段”从左到右访问各条带；两个“通道段”从右到左返回。五段之间各用一次向下移动衔接，其余只用 D 或向左的三个方向。

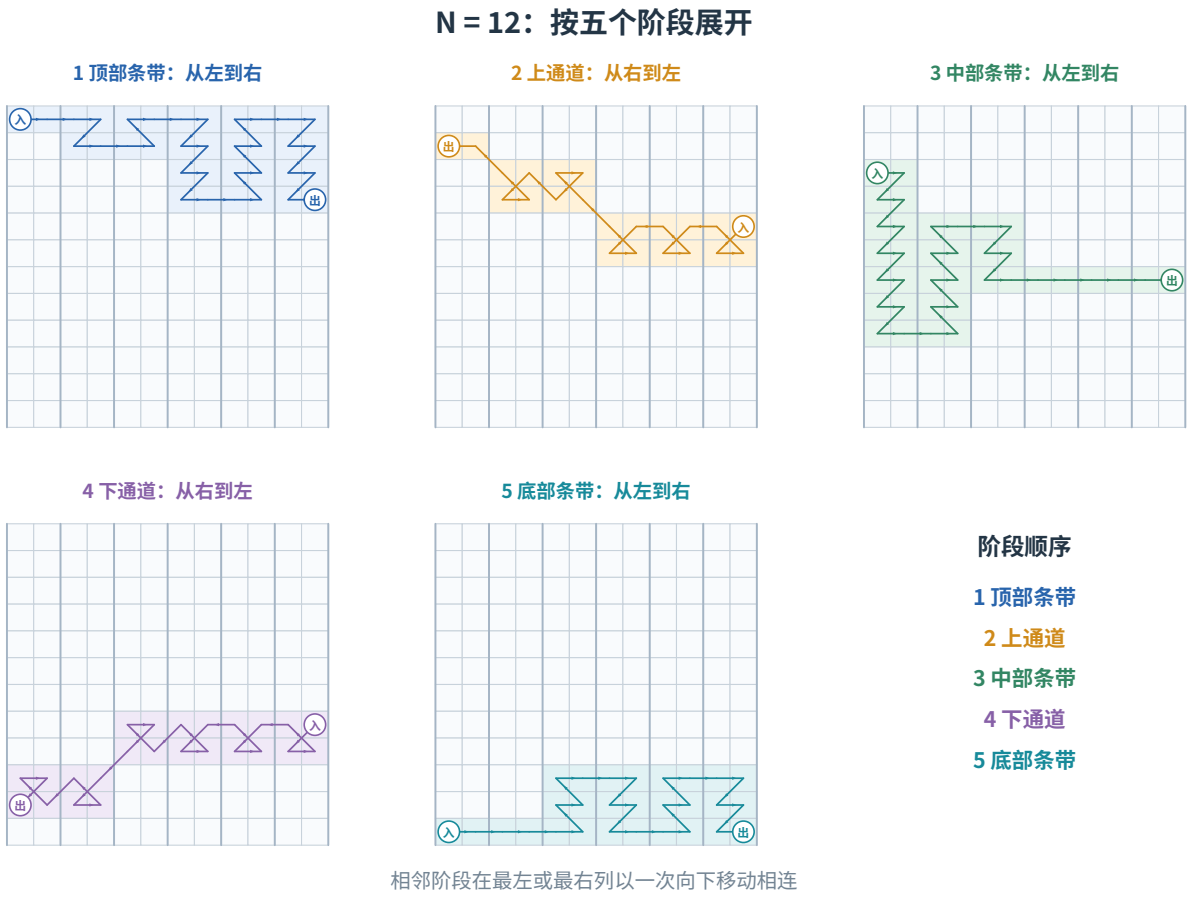


图 3　五阶段的实际走法。每一幅只画当前阶段，入、出标记表示本段的两个端点。

先准备两个小工具。 记第 j 个条带第 h 行的左右两格为

$$L_j(h) = (h, 2j), \qquad R_j(h) = (h, 2j+1).$$

`stripe(j,lo,hi,down)` 用来扫描这个条带的连续行：若 `down=true`，行号从 `lo` 增加到 `hi`，否则反过来；每行均依次加入 $L_j(h), R_j(h)$。

另一个工具是“折返模块”。给每条带指定 $d_j \in \{-1, 1\}$，定义

$$F(j,h):\quad R_j(h) \to L_j(h+d_j) \to R_j(h+d_j) \to L_j(h).$$

它占据相邻两行，从右格进入、从左格离开。$d_j = -1$ 时方向串为 `QDZ`，$d_j = 1$ 时为 `ZDQ`。模块内部没有竖直步，也没有右斜步。

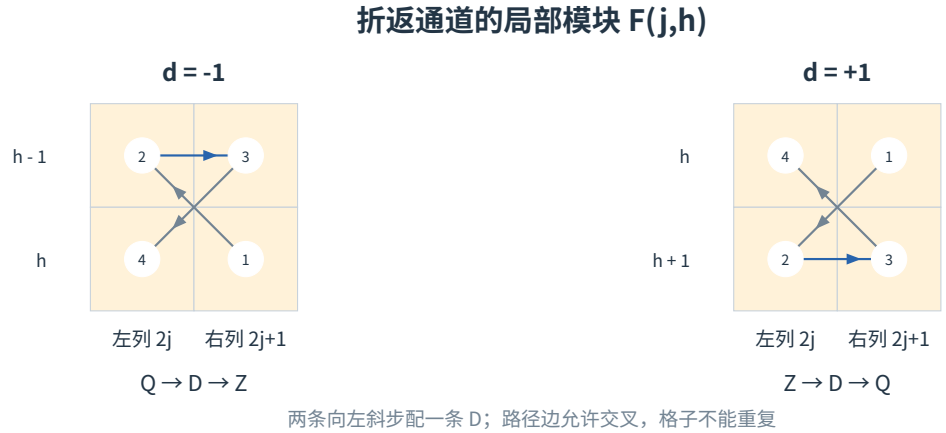


图 4　折返模块的两种方向。数字表示模块内部的访问顺序，两条斜边交叉是允许的。

然后划分每个条带的五个区域。 对 $j=0,\dots,m-1$，计算

$$a_j=\min(j+1,m-2),\qquad b_j=\max(N-2-j,m+1),$$

$$d_j=\begin{cases}-1, & j\ \text{为偶数且}\ j<m-3,\\ 1, & \text{其他}.\end{cases}$$

上通道使用第 a_j 行和它的折返邻行；下通道使用第 b_j 行和它的折返邻行。**上通道的 $j=0$ 是唯一例外：只走 $R_0(a_0)\to L_0(a_0)$，不使用邻行。**

为避免条带段走进通道，设 $[P]$ 在条件 P 成立时为 1，否则为 0，并计算四个端点：

$$\begin{aligned}t_j&=a_j-1-[j>0\wedge d_j=-1],\\ u_j&=a_j+1+[j>0\wedge d_j=1],\\ v_j&=b_j-1-[d_j=-1],\\ w_j&=b_j+1+[d_j=1].\end{aligned}$$

接下来按下表顺序向答案中追加格子即可。扫描一个条带时，“向下”指行号递增。

阶段	条带枚举顺序	追加内容
1 顶部	$j=0\to m-1$	扫描 $[0,t_j]$，j 为奇数时向下
2 上通道	$j=m-1\to 0$	$j>0$ 时追加 $F(j,a_j)$；$j=0$ 时只追加该行右格、左格
3 中部	$j=0\to m-1$	扫描 $[u_j,v_j]$，j 为偶数时向下
4 下通道	$j=m-1\to 0$	追加 $F(j,b_j)$
5 底部	$j=0\to m-1$	扫描 $[w_j,N-1]$，j 为奇数时向下

这些式子只是把“通道占用的行”从普通条带扫描中扣掉。例如 $d_j=1$ 时，上通道占用 a_j,a_j+1，所以中部从 a_j+2 开始；$d_j=-1$ 时，则将顶部的末行减一。

以 $N=12$ 为例：

$$\begin{aligned}(a_0,\dots,a_5)&=(1,2,3,4,4,4),\\ (b_0,\dots,b_5)&=(10,9,8,7,7,7),\\ (d_0,\dots,d_5)&=(-1,1,-1,1,1,1).\end{aligned}$$

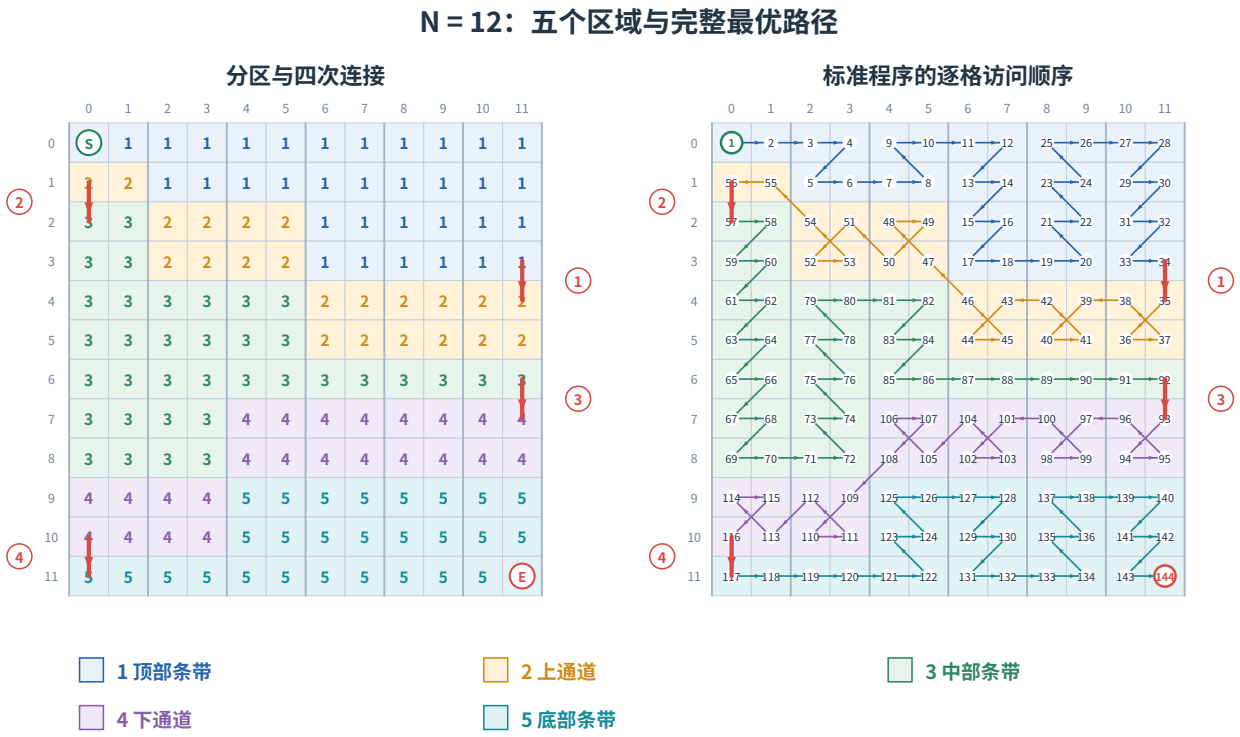


图 5　左图用颜色和编号区分五个区域；右图是同一构造的完整访问顺序。红色箭头标出四次阶段连接。

实现时，**五段依次加入同一个坐标数组即可，不用额外插入连接格子**。前一段的末格和后一段的首格已经相邻；重复加入端点反而会访问同一格两次。

四次连接的位置固定为：

连接	前一格	后一格
顶部到上通道	$(m-3, N-1)$	$(m-2, N-1)$
上通道到中部	$(1, 0)$	$(2, 0)$
中部到下通道	$(m, N-1)$	$(m+1, N-1)$
下通道到底部	$(N-2, 0)$	$(N-1, 0)$

所以恰好有四次 `X`，没有其他竖直步或右斜步，得到 $C = 4$。

3. 输出与复杂度

按照上述分支得到格子序列 $p_0, p_1, \ldots, p_{N^2-1}$。对相邻两格求行列差 $(\Delta r, \Delta c)$，用下表转成方向字符：

```
Q W E
A ? D
Z X C
```

表中行号为 $\Delta r + 1$、列号为 $\Delta c + 1$；`?` 不会被使用。每组先输出最优值，再输出方向串。

每格只加入一次，时间、空间均为 $O(N^2)$，与输出规模相同。正解对应 `std_maximum.cpp`。

4. 部分分路线

- **累计 10 分，$N \le 3$：** DFS 枚举路径，记录遍历全部格子并到达右下角时最大的 `D` 数。
- **累计 25 分，$N \le 4$：** 子集 DP，状态为“访问集合、最后一格”，保存最大 `D` 数并恢复路径。
- **累计 45 分：** 加上奇数的逐行蛇形。

- **累计 65 分：** 加上 $N \equiv 2 \pmod 4$ 的两列条带。
- **100 分：** 加上 $N = 8$ 特例和其余四的倍数的五阶段构造。

子集 DP 的时间、空间为 $O(2^{N^2} N^2)$。$N = 4$ 也可直接使用上面的固定构造。

5. 为什么达到最优

奇数。按列号奇偶染色，两种颜色的格子数相差 N，起终点都属于较多的颜色。要把这些格子串成一条路径，至少有 $N - 1$ 次相邻格子同色；只有竖直步保持列颜色，因此 $V \geq N - 1$。代入计数式得到 $D \leq (N^2 - 1)/2$，逐行蛇形达到等号。

模 4 余 2。 显然 $C \geq 0$，两列条带构造直接达到 $C = 0$。

四的倍数。C 必为偶数。若 $C = 0$，只允许 `D` 以及向左的三种步。第一列每格的出边都被强制为 `D`；将这些同行的左右格缩成一对，可知第一条带必须从顶到底走完，再依次交替扫描其他条带。条带数为偶数时，这条强制路径停在右上角，因此 $C = 0$ 不可能。

对于 $N = 4, 8$，上述固定构造达到 $C = 2$，已经最优。对于 $N \geq 12$，还需排除 $C = 2$：

- 若 $V = 0, R = 1$, 两列条带内的右向边必须形成完美匹配, 不能仅有一条右斜边; 把它放在条带间，又会让下一条带从内部行进入，无法遍历所有行。
- 若 $V = 2, R = 0$, 可由列切口计数确定一段连续的“活动条带”。每条活动条带恰有一行缺少内部 `D`，记为 h_j。相邻缺口必须相邻，即 $|h_{j+1} - h_j| \leq 1$。顶端和底端的边界状态约束进一步要求 $\min h_j \leq 2$、$\max h_j \geq N - 3$，于是活动条带数至少为 $N - 4$，但总条带数只有 $N/2$，推出 $N \leq 8$，矛盾。

因此 $N \geq 12$ 时至少需要 $C = 4$，而五阶段构造恰好达到它。上面第二种情况的完整切口推导和边界状态分析见附件 PDF “最优值证明”。

最后检查五阶段构造的合法性：两条通道占据的行先被预留，其余行恰好分给顶部、中部、底部三个连续段，所以没有遗漏或重复。条带段内部、相邻条带之间及通道内部的连接均为八邻接；五段之间的连接已在表中列出。首格为左上角，末格为右下角，因此确实得到一条合法最优路径。

存储空间 题解

下面按 std_hld.cpp 的实现顺序讲解。完整算法为 $O(n+q\log^2 n)$ 时间、$O(n)$ 空间；先弄清“改哪些需求”和“需求如何影响 DP”，再把两部分接起来。

1. 先把原区间变成节点需求

对目标区间 $[l,r]$，取完整包含于它、且不能继续向父亲扩大的树节点。这些节点称为规范分解。例如八个叶子的完全二叉树上，$[2,7]$ 分成

$$[2,2],\quad [3,4],\quad [5,6],\quad [7,7].$$

满足这个目标，等价于分别完整覆盖这些块。维护

$$c_v = \text{当前有多少个原区间的规范分解包含节点 } v.$$

DP 只关心 c_v 是否为零，但增删时必须保存完整次数。例如四个叶子的完全二叉树上，$[1,3]$ 与 $[1,2]$ 都会需求节点 $[1,2]$；删除前者后，这个节点仍有一次引用。

一次规范分解可能有线性多个块，不能直接枚举。关键是：**除了左右端点所在的块，其余块都是两条边界路径旁的兄弟子树**。把这些兄弟的计数放在路径上父亲的位置，就能用重链区间修改。

2. 先写出能重算的树形 DP

每个状态用 `Value{cost, ways}` 保存最小费用及其方案数。两个独立部分合并时，费用相加、方案数相乘，记为 $\otimes$；两个互斥方案择优时，保留较小费用，平局则加方案数，记为 $\oplus$。方案数模 998244353。

状态	需要满足什么
X_v	子树内所有严格子孙的需求，暂不要求覆盖整个 I_v
B_v	同样满足子孙需求，并且完整覆盖 I_v
A_v	实际使用的状态：$c_v=0$ 取 X_v，否则取 B_v

叶子有 $X_v=(0,1)$、$B_v=(w_v,1)$。内部节点的左右孩子记为 L,R：

$$X_v = A_L \otimes A_R, \qquad B_v = \big((w_v,1)\otimes X_v\ \big)\oplus (B_L\otimes B_R).$$

`B` 的两个候选分别是“购买 v，子孙需求另外满足”和“不购买 v，完整覆盖两个孩子”。购买一个祖先不能代替对较小目标的精确覆盖，所以第一项仍然要加上 X_v。

例如父节点费用为 2，两个叶子费用均为 1。只需求父区间时，选父亲、选两个叶子都是费用 2，共有两种方案；再需求左叶子后，选父亲还得加买左叶子，费用变成 3，最优只剩两个叶子这一种。这个例子同时说明了祖孙需求与平局计数。

实现里不必保存 `A` 数组，用到孩子时根据其计数选择 `X/B` 即可。尤其要让 $\boldsymbol{X_v, B_v}$ **都不依赖** $\boldsymbol{c_v}$：改变 c_v 只改变父亲怎样使用它，不必重算 v 的整棵子树。根没有父亲，答案单独取 `rootCount ? B[1] : X[1]`。

3. 把重链上的 DP 写成矩阵

按子树叶子数选择重孩子，平局选右孩子。设 v 的重孩子为 h、轻孩子为 s，先把轻子树贡献记为

$$C = (c_s = 0\ ?\ X_s : B_s).$$

先固定轻孩子的贡献 C, B_s。重孩子无需求时，$X_v = C\otimes X_h$；有需求时，$X_v = C\otimes B_h$。再把 B_v 的转移按 X_h, B_h 归类，便得到矩阵的第二行：

$$\begin{pmatrix} X_v \\ B_v \end{pmatrix} = M_v \begin{pmatrix} X_h \\ B_h \end{pmatrix}, \qquad M_v = \begin{cases} \begin{pmatrix} C & \mathbf{0} \\ (w_v,1)\otimes C & B_s \end{pmatrix}, & c_h = 0, \\ \begin{pmatrix} \mathbf{0} & C \\ \mathbf{0} & ((w_v,1)\otimes C)\oplus B_s \end{pmatrix}, & c_h > 0. \end{cases}$$

其中 $\mathbf{0} = (\mathrm{INF}, 0)$ 表示不可行。矩阵乘法里的加、乘分别用 $\oplus, \otimes$，对应代码 `add`、`mul`。有限费用即使方案数取模后为零，也仍然可行。

一条链的矩阵从链头向链尾相乘。链尾一定是叶子 t，让它存单位矩阵，整条链乘积再作用于 $((0,1),(w_t,1))^T$，就得到链头的 `X/B`。查询与合并必须保持这个顺序。

代码中的主要字段如下：

字段	作用
`low/high, leafId`	节点的叶序区间、叶序到节点的映射
`heavy/light, head, dfn, at, tail`	重轻孩子、链头、链内连续编号、编号反查、链尾
`X/B`	动态传播时只需更新链头状态；轻孩子一定是另一条链的链头
`heavyCount[v]`	重孩子的需求次数，即 $c_{heavy[v]}$
`rootCount`	根自身的需求次数

`transition(v, lightPositive)` 就是上面的矩阵：重孩子状态由 `heavyCount[v]` 决定，轻孩子为零、为正的两种矩阵都能随时构造。

4. 将需求计数放到父亲处

每个非根节点的需求只保存一份：

- 重孩子的计数存在父亲的 `heavyCount` 中；修改后仅当零/正状态切换时重建父矩阵。
- 轻孩子的计数放在线段树的 `dfn[parent]` 位置，由线段树维护。
- 根的计数存在 `rootCount` 中。

因此修改某个规范节点时，`addNode(v, delta)` 为：

```
p = parent[v]
若 p = 0：rootCount += delta
否则若 heavy[p] = v：修改 heavyCount[p]，必要时 pointRemake(dfn[p])
否则：rangeAdd(dfn[p], dfn[p], 轻孩子方向, delta)
```

方向约定为 `0`=左，`1`=右；父亲位置 `p` 的轻孩子方向即 `light[p] == rc[p]`。一条链内，路径经过重边时，所需的兄弟一定是轻孩子：把对应父亲位置做区间加减，并只修改指定方向。路径跨轻边时，兄弟变成重孩子，改一次 `heavyCount`。每条根路径只有 $O(\log n)$ 条轻边，点修改数量也就很少。

5. 线段树怎样一次改变很多矩阵

轻孩子的计数可能在一整段上同时从零变正，或从正变零。不能逐个重建矩阵。每个线段树节点保存代码中的三组字段：

```
Matrix product[4];
int minimum[2], lazy[2];
```

对每个方向 d，`minimum[d]` 是这一段该方向计数的最小值。没有这个方向的位置时，用 `NO_COLOR` 表示不存在。所有计数非负，所以最小值为零时，恰好那些取最小值的位置需要使用“轻孩子计数为零”的矩阵。

`product[mask]` 保存一段矩阵乘积，`mask` 的两个二进制位分别控制两种方向：

- 第 d 位为 `0`：该方向全部使用正计数矩阵；
- 第 d 位为 `1`：该方向取最小值的位置使用零计数矩阵，其余仍用正计数矩阵。

例如一种方向的计数为 $[0,2,0]$，对应位为 `1` 的乘积使用“零、正、零”三种矩阵。整段加一后计数变为 $[1,3,1]$，取最小值的位置没变，保存的假想乘积也没变；只需把真实选择切换到该位为 `0` 的乘积。

整段加减：tag 和 push

整段同方向加 delta 不改变最小值位置，因此只改两个整数，不改任何乘积：

```
tag(p, d, delta):
    若 minimum[p][d] = NO_COLOR：返回
    minimum[p][d] += delta
    lazy[p][d] += delta
```

下放时，把两个方向各自的 lazy 用同一个 tag 加给孩子，再清零。合法删除保证计数不会减成负数。

合并：pull

先分别求父区间的两方向最小值。某个孩子只有在自己的最小值等于父最小值时，才含有“父区间最小值位置”，允许继承对应的模式位。

```
minimum[p][d] = min(minimum[left][d], minimum[right][d])
eligibleLeft  = 两个方向上，左孩子最小值等于父最小值的位集合
eligibleRight = 两个方向上，右孩子最小值等于父最小值的位集合
对 mask = 0..3：
    product[p][mask] = product[left][mask & eligibleLeft]
                     × product[right][mask & eligibleRight]
```

例如左孩子最小值为 0、右孩子为 2，父模式要求最小值位置用零矩阵时，只能给左孩子这一模式；右孩子必须使用全正模式。某方向不存在时，其模式位无作用，代码会直接复用清除该位后的乘积。

查询完整维护区间时，真实模式是

```
mask = (minimum[0] == 0) | ((minimum[1] == 0) << 1)
```

返回 product[mask]；部分覆盖时递归查询并按从上到下的顺序相乘。

单点重建：pointRemake

重孩子计数过零，或者轻子链的 X/B 改变时，需要重建父亲位置。先沿途 push，到叶子后用 transition(v, false/true) 重建四种矩阵，再一路 pull。**保留这个位置已有的轻孩子计数与懒标记含义，不能重置为零**。原树的叶子没有孩子，四种乘积都设为单位矩阵。

6. 一次增删操作怎样接起来

先找端点叶子的最高包含祖先：

```
a = maximal(leafId[l], l, r)
b = maximal(leafId[r], l, r)
```

maximal 从当前链向上跳：若链头区间包含于 $[l, r]$，就尝试链头父亲；否则在链头到当前节点的 dfn 区间中，二分第一个完整包含的节点并返回。完整包含的条件是 low[v] >= l && high[v] <= r。只需最后一条链内二分一次。

若 $a = b$，只给它加一次计数。否则令 $z = LCA(a, b)$，除 a, b 外：左边界上补右兄弟，右边界上补左兄弟，**排除父亲恰为 z 的边**。前面的 $[2, 7]$ 例子中，两侧分别补出 $[3, 4]$ 与 $[5, 6]$。

下面是与代码一致的边界更新。rangeAdd(L,R,d,delta) 在 $L > R$ 时不做事，side 返回孩子位于左侧还是右侧。

```
boundary(v, z, d, delta):
    while depth[v] > depth[z] + 1:
        h = head[v]
        if h == head[z]:
            rangeAdd(dfn[z]+1, dfn[v]-1, d, delta)
            return
        rangeAdd(dfn[h], dfn[v]-1, d, delta)
        p = parent[h]
        if depth[h] > depth[z]+1 and side(heavy[p]) == d:
            addHeavy(p, delta)
        v = p
```

链内区间修改的是经过重边时的父亲位置；跨链处单独处理边 (p, h)。两个 `depth` 条件都用于排除 LCA 边。

直接改完这些计数后，还要把变化传回根。不要每做一次区间修改就单独向根传播。所有受影响矩阵都在 `parent[a]`、`parent[b]` 到根的两条路径上，先收集这些路径的链头，再统一处理：

```
update(op, l, r):
    delta = (op == '+' ? 1 : -1)
    a = maximal(leafId[l], l, r)
    b = maximal(leafId[r], l, r)
    chains = parent[a] 到根经过的链头
    若 a != b：再加入 parent[b] 到根经过的链头

    addNode(a, delta)
    if a != b:
        addNode(b, delta)
        z = lca(a, b)
        boundary(a, z, 1, delta)   // 左边界补右兄弟
        boundary(b, z, 0, delta)   // 右边界补左兄弟

    将 chains 去重，按链头深度从大到小排序
    for h in chains:
        P = query(dfn[h], dfn[tail[h]])
        (newX, newB) = P × ((0,1), (weight[tail[h]],1))^T
        若 newX/newB 的费用和方案数都未变：继续
        X[h] = newX; B[h] = newB
        若 parent[h] != 0：pointRemake(dfn[parent[h]])

    输出 rootCount > 0 ? B[1] : X[1]
```

深链先完成，所以处理父链时，其轻子链已经报告了新状态。若此次只改根的计数，`chains` 为空，直接重新选择根的 `X/B` 即可。

7. 初始化与复杂度

先按真实树形做显式栈遍历，建立父亲、深度、从左到右的叶序；再逆序计算区间、叶子数及重轻孩子。节点编号任意，不能把编号倒序当作后序。

初始计数全为零，所有 $X_v = (0, 1)$；叶子 $B_v = (w_v, 1)$，内部节点计算

$$B_v = (w_v, 1) \oplus (B_L \otimes B_R).$$

随后连续编号每条重链，确定 `head/dfn/at/tail`，在线段树各位置初始化方向最小值为零、不存在方向为 `NO_COLOR`，构造四种乘积。原树可能是长链，预处理使用迭代遍历；线段树递归深度只有对数级。

一次操作涉及 $O(\log n)$ 个链区间、点修改及待传播链，每项花 $O(\log n)$，因此总时间 $O(n + q \log^2 n)$，空间 $O(n)$（$n = 1$ 时直接维护唯一节点需求）。费用用 `long long`，乘方案数用 64 位中间量。

8. 部分分路线

各档独立计分，按限制可逐步省去维护结构：

1. **5 分，$n \le 7, q \le 100$：** 枚举存储节点集合，用区间拼接可达性判断覆盖。
2. **10 分，$nq \le 2 \times 10^6$：** 枚举规范块，每次全树重算 DP，$O(nq)$。
3. **10 分，精确中点分割：** 沿区间边界递归更新 DP，$O(n + q \log n)$。
4. **5 分，叶深不超过 30：** 同样沿实际边界更新，$O(n + 30q)$。
5. **10 分，目标都是节点区间：** 计数点修改加普通动态树 DP，无须方向区间更新。
6. **10 分，左孩子均为叶子：** 内部节点只有一条链、一种方向，维护两种乘积即可。
7. **10 分，右子树不少于左：** 只有左侧轻孩子，保留多链传播，乘积只需两种。
8. **10 分，仅插入：** 有序集合跳过已激活的轻孩子位置，每点首次激活后删除，配合普通点修改动态树 DP。
9. **5 分，长度至多 2：** 按双叶组件计算局部费用与平局数，每次只改常数个组件，$O(n + q)$。
10. **25 分，无额外限制：** 使用完整算法。

第 8 档的所有激活方式须共用同一份标记；轻孩子若由端点点操作激活，也要从未激活集合删除。第 9 档跨组件的长度二目标会产生两个叶子需求，仍需维护引用次数。

附录：为什么上述维护正确

规范分解中的块两两无交。任何能参与目标精确覆盖的树节点，都落在唯一一个规范块内，所以覆盖原目标等价于覆盖全部规范块。最高包含祖先确定两端块，边界路径旁的内侧兄弟恰好补齐中间部分，排除 LCA 边后没有重复。

在每棵子树内，`X` 只合并两个孩子的实际需求；`B` 按是否购买当前节点分成两个互斥集合。正费用保证 `X` 不会额外购买当前节点，两个分支择优与计数均正确。矩阵只是同一递推的展开。

合法操作保持计数非负，零位置恰好由最小值判定；整段平移不改变最小值位置，四种假想乘积和 `pull` 的资格筛选因而始终有效。最后按深度处理链，每次都使用已更新的轻子链状态，根得到的就是当前全部需求的最优费用与方案数。